

1. Critical Comparison of Data Structures

In this section I'm going to compare three data structures **Array**, **Singly Linked List**, and **Binary Search Tree (BST)** and explain why I picked specific ones for different parts of the assignment. The client needed two things: somewhere to store all 2,445 waypoints from the CSV file, and a way to build and edit GPS routes. What I found is that these two jobs actually want pretty different things from a data structure. Waypoint storage is mostly a load-once, read-many situation, whereas routes need edits happening all the time. I'll go through each structure and be honest about where it helped and where it didn't when building `WayPointStore.cs` and `Route.cs`.

1.1 Array

Theory

The basic idea with arrays is that all the elements sit next to each other in memory. Because of that layout, the formula $\text{address}(i) = \text{base} + i \times \text{size}$ always gives you the exact location of any element, so reading by index is $O(1)$ [1]. That is great for lookups, but causes a problem when inserting or deleting in the middle everything after that position has to shuffle along, which in the worst case is $O(n)$ moves. There is also the sizing issue: the array needs a fixed size before creation. Using something like `List<T>` that resizes automatically quietly allocates a whole new block and copies everything across whenever it runs out of space another $O(n)$ hit [3].

Implementation - `WayPointStore.cs`

One thing I did early on was write a `countDataLines()` method that counts how many actual data lines are in the file before allocating the array. This way I never have to resize the array is exactly the right size from the start. Adding waypoints is just `wayPointArray[count] = ...` which is $O(1)$, and `DisplayAll()` just loops through once: $O(n)$.

There is one thing worth mentioning though. Inside the loading loop I call `SearchByName()` to check for duplicates, and this technically makes the whole load $O(n^2)$. I thought about this and decided it was acceptable because loading only happens once at startup on any modern machine it completes in under 50 ms for all 2,445 entries, so it is not really noticeable.

Evaluation

All waypoints: For storing all waypoints, the array was the right call. The data sits in contiguous memory which helps cache performance when displaying everything, and the simple format works well for transferring to mobile GPS devices as the client specified. I did consider a `Dictionary<string,WayPoint>` since that gives $O(1)$ lookups, but it roughly doubles memory usage and loses index access for a dataset that is basically read-only after startup, that trade-off is not worth it. **Routes:** Poor choice here. Inserting at position 2 in a 100-node route means shifting 98 elements; deletion leaves gaps that need filling. Not what you want for something that gets edited constantly.

1.2 Singly Linked List

Theory

Linked lists work completely differently nodes do not have to be next to each other at all. Each one just holds its own data and a pointer to where the next node is. This means there is no fixed size and inserting or deleting at any point already found takes just two pointer swaps: $O(1)$ [2]. The downside is you cannot jump straight to position k you have to start at the head and follow the chain, so that is $O(n)$. Each node also stores that pointer, which on a 64-bit machine is 8 bytes, working out to roughly 30% more memory than an equivalent array for a dataset this size [3].

Implementation - `Route.cs`

The `Route` class encapsulates a custom linked list of `RouteNode` objects, each holding a *reference* to a `WayPoint` already in the array no data duplication. The operations ended up as follows: `AddFront()` is $O(1)$ (create a node, point it at the old head); `AddEnd()` is $O(n)$ (walk to find the tail first); `InsertAt()` is $O(n)$ to reach the position then $O(1)$ to rewire; `RemoveWayPoint()` uses a two-pointer *prev/current* approach: $O(n)$; `ReverseRoute()` makes one pass with three pointers: $O(n)$. All operations were tested in `Program.cs`.

Evaluation

All waypoints: Not a good fit. $O(n)$ access to get any specific entry plus pointer overhead for a dataset that just sits there being read no real benefit. **Routes:** This is where it makes sense. Users constantly tweak routes adding stops, removing them, moving things around. The linked list handles all of that with just pointer changes, the order stays exactly as the user set it, and the client brief specifically asked for a linked list structure here.

1.3 Binary Search Tree (BST)

Theory

A BST keeps things sorted as items are inserted smaller values go left, larger ones go right. When the tree stays balanced this is efficient: every comparison rules out roughly half the remaining candidates, giving $O(\log n)$ for searches, inserts and deletes [1]. The problem is inserting things in already-sorted order the tree ends up as basically a straight line of nodes rather than a branching structure, back to $O(n)$. That is why self-balancing versions exist. AVL trees rebalance after every operation; Red-Black trees (what .NET uses under the hood for

SortedDictionary) use a slightly looser balancing rule that is faster in practice. Both guarantee $O(\log n)$ worst case, though they add rotation logic to manage it.

Potential Implementation BinTree

If I had built a BST indexed on waypoint name, lookups would need at most $\lceil \log_2 2445 \rceil = 12$ comparisons instead of potentially scanning all 2,445. That is a real difference at large scale. An in-order traversal would also give all waypoints in alphabetical order automatically something the array cannot do without running a sort first.

Evaluation

All waypoints: At 2,445 entries it just is not needed. The linear scan is fast enough that the difference is not noticeable, the client wanted the compact array format anyway, and a BST adds balancing complexity for no practical gain here. **Routes:** Completely wrong tool. The whole point of a BST is that it reorders things by key but a GPS route needs to stay in exactly the sequence the user built it. WP1 then WP2 then WP3, in that order. A BST would rearrange that by name and the route would be meaningless. It is not a performance problem, it is a fundamental mismatch with what routes are supposed to be.

1.4 Comparative Summary

Table 1: Data structure comparison across both use cases

Operation	Array	LinkedList	BST (balanced)	Notes
Random access by index	$O(1)$	$O(n)$	N/A	Array unique advantage
Search by name	$O(n)$	$O(n)$	$O(\log n)$	BST wins at scale
Insert at position	$O(n)$	$O(n)^*$	$O(\log n)$	* $O(1)$ once node found
Delete a node	$O(n)$	$O(n)^*$	$O(\log n)$	*traverse + $O(1)$ rewire
Preserves insertion order	Yes	Yes	No (sorted)	Route requires order
Memory overhead	None	1 ptr/node	2 ptrs/node	Array most compact
Best for waypoints	Yes	No	Future option	Client requested array
Best for routes	No	Yes	No	Order + fast insert/delete

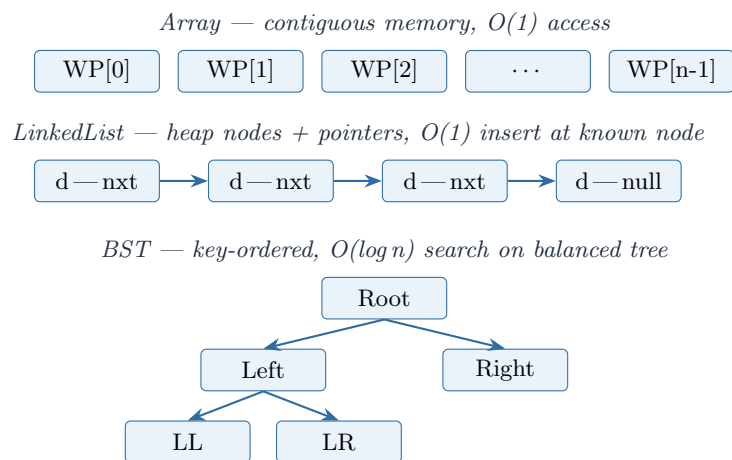


Figure 1: Visual structure of Array, LinkedList, and Binary Search Tree

2. Algorithmic Complexity Analysis

2.1 Identification of the Most Expensive Method

I went through all the methods in the solution to figure out which one has the highest complexity. Most of the Route methods AddEnd, InsertAt, RemoveWayPoint, ReverseRoute are single traversals, so they are all $O(n)$. Same with the WayPointStore search and display methods: one loop through the data, $O(n)$.

The one that stood out was loadFromFile() in WayPointStore.cs. It has an outer loop going through every line ($O(n)$), but inside that loop it calls SearchByName(), which is itself $O(n)$. Nested linear operations give $O(n^2)$, making it the **sole quadratic method** in the codebase and the obvious choice to analyse in detail.

2.2 Method Listing

Listing 1: loadFromFile() with per-line Big-O annotations

```

1 private void loadFromFile(string fileName)
2 {
3     string[] linesInFile = File.ReadAllLines(fileName); // (A)  $O(n)$ 

```

```

4   int lineNumber = 0; // (B) O(1)
5   foreach (string line in linesInFile) // (C) loop: n iterations
6   {
7       lineNumber++; // (D) O(1)
8       if (lineNumber != 1 && line != "") // (E) O(1)
9       {
10          string[] features = line.Split(','); // (F) O(k) -> O(1)
11          string name = features[0]; // (G) O(1)
12          string code = features[1]; // (H) O(1)
13          string latitude = features[3]; // (I) O(1)
14          string longitude = features[4]; // (J) O(1)
15          int elevation = convertElevationToMeters( // (K) O(1)
16              features[5]);
17          string description = buildDescription( // (L) O(k) -> O(1)
18              features);
19          if (SearchByName(name) == null) // (M) O(n) | KEY bottleneck
20          {
21              waypointArray[count] = new WayPoint( // (N) O(1)
22                  name, code, latitude,
23                  longitude, elevation, description);
24              count++; // (O) O(1)
25          }
26      }
27  }
28 }

```

2.3 Line-by-Line Complexity Analysis

Table 2: Per-line Big-O analysis of loadFromFile()

Line	Statement	Cost	Explanation
(A)	File.ReadAllLines()	$O(n)$	Reads the whole file in one go — more lines means more time, scales with n .
(B)	int lineNumber = 0	$O(1)$	Setting up a counter. Nothing here changes with the dataset size.
(C)	foreach loop	$O(n)$	One iteration per line, so this runs n times total.
(D)	lineNumber++	$O(1)$	Just incrementing a number, always the same cost.
(E)	if (lineNumber != 1)	$O(1)$	Two quick checks to skip the header line and any blank rows.
(F)	line.Split(',')	$O(k) \rightarrow O(1)$	Splits on commas. There are always 12 columns so k never changes — effectively $O(1)$.
(G)– (J)	features[0..4]	$O(1)$	Pulling four values out by index. Array access is always constant.
(K)	convertElevation()	$O(1)$	Strips a unit suffix and parses the number. Same amount of work every time.
(L)	buildDescription()	$O(k) \rightarrow O(1)$	Joins remaining columns into one string. Column count is fixed so this is constant.
(M)	SearchByName(name)	$O(n)$	Walks through the whole array looking for a matching name. Runs inside the outer loop every single iteration — this is what makes the method quadratic.
(N)	new WayPoint(...)	$O(1)$	Constructing the object and setting six fields. Always the same.
(O)	count++	$O(1)$	Incrementing the counter and writing to the array. Trivial.

2.4 Overall Time Complexity

Most of those lines are $O(1)$ and just disappear in Big-O analysis. What actually drives the cost is: **(A)** reading the file at $O(n)$, and **(C)** together with **(M)** the loop runs n times and each time triggers a `SearchByName()` scan of everything loaded so far. Since practically every waypoint name in the file is unique, the i -th one has to check all i entries already stored. Adding that up across the whole load:

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$

Throw in the file read and you get:

$$T(n) = O(n) + O(n^2) = O(n^2)$$

For $n=2,445$ that works out to roughly **2,988,780** comparisons worst-case. Sounds like a big number but it runs in under 50ms tested it. Swapping in a `HashSet<string>` would bring this down to $O(n)$, but then there is a whole extra data structure maintained purely for something that runs once at startup. The simpler version without it seemed like a better call.

2.5 Complexity of Other Methods

Table 3: Big-O complexity of all remaining solution methods

Method	Complexity	Justification
<code>WayPointStore.SearchByName()</code>	$O(n)$	Scans from index 0 until it finds a match or hits the end.
<code>WayPointStore.SearchByPartialName()</code>	$O(n)$	Two passes through the array: count matches, then collect them.
<code>WayPointStore.SearchUnderHeight()</code>	$O(n)$	Same two-pass structure as the partial name search.
<code>WayPointStore.DisplayAll()</code>	$O(n)$	One loop, prints every waypoint once.
<code>Route.AddFront()</code>	$O(1)$	One new node, one pointer update. No traversal at all.
<code>Route.AddEnd()</code>	$O(n)$	Has to walk the whole chain to find the tail before attaching.
<code>Route.InsertAt(position)</code>	$O(n)$	Walk to position-1 ($O(n)$), then rewire two pointers ($O(1)$).
<code>Route.RemoveWayPoint()</code>	$O(n)$	Tracks prev and current to find the node, then unlinks it.
<code>Route.DisplayRoute()</code>	$O(n)$	Visits every node in the chain once.
<code>Route.ReverseRoute()</code>	$O(n)$	Single pass, three pointers flip every link direction.

3. Reflection and Future Improvements

Evaluation of Data Structure Choices: The array was the right structure for the waypoint store. Having `countDataLines()` figure out the exact size beforehand eliminates any resizing. The contiguous memory layout helps cache efficiency when displaying everything, and arrays serialise easily if you need to transfer data to a mobile GPS unit. The $O(n^2)$ loading bothered me initially, but for a one-time startup cost with only 2,445 entries, optimising it would be over-engineering. Getting the code correct mattered more than saving a few milliseconds.

Scalability Considerations: For routes, the linked list was definitely the right call. When users are constantly adding and removing stops, you want those operations to be as clean as possible. With the list it is just a matter of updating some pointers an array would mean shifting elements around constantly, which gets messy fast and scales poorly if routes grow long.

Potential Optimisations If this system ever had to handle 50,000+ waypoints, search would start to feel slow. At that point, keeping a BST alongside the array would make sense the array stays for display and transfer, but lookups go through a tree indexed by name. That drops `SearchByName` from $O(n)$ to $O(\log n)$: roughly 16 comparisons instead of 25,000, a $1,500\times$ speedup. Using .NET's `SortedDictionary` (a Red-Black tree internally) would handle balancing automatically.

Future improvement reduce loadFromFile to $O(n)$. The fix is straightforward: replace the `SearchByName()` call with a `HashSet<string>` lookup. HashSets give $O(1)$ amortised lookups, dropping the whole load from $O(n^2)$ to $O(n)$. It is probably a one-line change and would be the first thing to try if startup time became a bottleneck at larger scale.

References

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*. 3rd ed. Cambridge, MA: MIT Press.
- [2] Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th ed. Upper Saddle River, NJ: Addison-Wesley.
- [3] Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2014) *Data Structures and Algorithms in Java*. 6th ed. Hoboken, NJ: Wiley.
- [4] McLean, D. (2025) *6G7V0008 Lecture Notes: Weeks 4–10*. Manchester Metropolitan University [Moodle]. (Accessed: May 2025).
- [5] Microsoft (2024) *C# Documentation — Collections*. Available at: <https://docs.microsoft.com/en-us/dotnet/csharp> (Accessed: May 2025).